



安徽大學
Anhui University

《JAVA程序设计》

第五章 进一步讨论类和对象

🎤 主讲人：黄小平

本章目录



安徽大學
Anhui University

1 抽象数据类型

2 对象的构造和初始化

3 子类

4 方法重写

5 多态

6 JAVA包

7 类成员

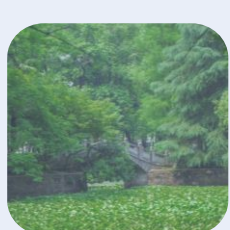
8 关键字final

9 抽象类

10 接口

11 内部类

12 包装类



5.1 抽象数据类型



安徽大學
Anhui University

抽象数据类型

- 早期程序设计语言中，复合数据类型及其相关的操作的代码之间没有特殊的联系。
- 抽象数据类型是指基于一个逻辑类型的数据类型以及这个类型上的一组操作。每一个操作由它的输入、输出定义。抽象数据类型的定义并不涉及它的实现细节，这些实现细节对于抽象数据类型的用户是隐藏的。
- 程序5-1给出了Date类型和tomorrow操作间建立的一种联系；

5.1 抽象数据类型



安徽大學
Anhui University

抽象数据类型

- ❑ 定义抽象数据类型后，需要为这个类型的对象定义操作，也就是方法。
- ❑ 定义方法的格式如下：

〈修饰符〉〈返回类型〉〈名字〉(〈参数列表〉)〈块〉

```
public static void main( string args) { }
```

- 〈名字〉是方法名，它必须使用合法的标识符。
- 〈返回类型〉说明方法返回值的类型。
- 〈修饰符〉段可以含几个不同的修饰符，其中限定访问权限的修饰符包括public, protected和private。
- 〈参数列表〉是传送给方法的参数表。表中各元素间以逗号分隔，每个元素由一个类型和一个标识符组成。
- 〈块〉表示方法体，是要实际执行的代码段。

5.1 抽象数据类型



- [程序5-2](#) Date类中增加daysInMonth()和printDate()方法

A screenshot of a Windows command prompt window titled "选定 命令提示符". The window shows the following commands and output:

```
D:\java\program\chapter5\prog5-2>javac Date.java  
D:\java\program\chapter5\prog5-2>java Date  
The current date is <dd / mm / yy>: 1/1/1998  
Its tomorrow is <dd / mm / yy>: 2/1/1998  
The current date is <dd / mm / yy>: 28/2/1964  
Its tomorrow is <dd / mm / yy>: 29/2/1964  
D:\java\program\chapter5\prog5-2>
```

5.1.3 按值传送



- ❑ “按值” 传送自变量，即方法调用不会改变自变量的值
- ❑ 当对象实例作为自变量传送给方法时，自变量的值是对对象的引用，也就是说，传送给方法的是引用值。在方法内，这个引用值是不会被改变的，但可以修改该引用指向的对象内容。因此，当从方法中退出时，所修改的对象内容可以保留下来。
- ❑ 程序5-3 创建pt对象，方法内局部变量val赋初值11。调用方法changeInt()后，val的值没有改变。字符串变量str作为changeStr的参数传入方法内，当从方法中退出后，其内容也没有变化。当对象pt作为参数传给changeObjValue()后，该引用所保存的地址不改变，而该地址内保存的内容可以变化，因此退出方法后，pt对象中的ptValue改变为99f

5.1.4 重载方法名



安徽大學
Anhui University

现在假定需要打印int、float和String类型的值。

```
public void printInt()  
public void printFloat()  
public void printString()
```



面向对象的程序设计：允许多个方法使用同一个方法名。

```
public void print(int i)  
public void print(float f)  
public void print(String s)
```

5.1.4 重载方法名



- 重载：允许对多个方法使用同一个方法名的现象，称为重载。
- 如果需要在同一个类中写多个方法，让它们对不同的变量进行同样的操作，就需要重载方法名
 - 一个方法区别于另一个方法的要素：方法名、参数列表及返回值。根据参数来查找适当的方法并调用，包括参数的个数及类型。
- 重载方法的两条规则：
 - 调用语句的自变量列表必须足够判明要调用的是哪个方法。
 - 重载方法的参数列表必须不同，方法的返回值类型可能不同；

5.2 对象的构造和初始化



- 说明了一个引用后，要调用new为新对象分配空间，也就是要调用构造函数
- 在Java中，使用构造函数（**constructor**，也称为构造方法）是生成实例对象的**唯一**方法。在调用new时，既可以带有变量，也可以不带变量，这要视具体的构造方法而定。
- 调用构造方法时步骤如下：
 - (1) 分配新对象的空间，并进行缺省的初始化。在Java中，这两步是不可分的，从而可确保不会有没有初值的对象。
 - (2) 执行显式的成员初始化。
 - (3) 执行构造方法，构造方法是一个特殊的方法。

5.2.1 显式成员初始化



安徽大學
Anhui University

- 在成员说明中写有简单的赋值表达式，就可以在构造对象时进行显式的成员初始化。

```
public class Initialized {  
    private int x = 5;  
    private String name = "Fred";  
    private Date created = new Date();  
    ...  
}
```

5.2.2 构造方法



□ 显式初始

定的初值

□ 构造方法

它的名字

✓ 为了

即构造

✓ 构造

父类

```
public class Xyz {  
    // 成员变量  
    int x;  
    public Xyz() { // 参数表为空的构造方法  
        // 创建对象  
        x = 0;  
    }  
    public Xyz(int i) { // 带一个参数的构造方法  
        // 使用参数创建对象  
        x = i;  
    }  
}
```

在创建Xyz的实例时，可以使用两种形式：

```
Xyz Xyz1 = new Xyz();  
Xyz Xyz2 = new Xyz(5);
```

性。（设

的功能。

自动调用。

方法，

不能从

5.2.2 构造方法



安徽大學
Anhui University

□ 构造方法的特性：

- ✓ 构造方法的名字与类名相同
- ✓ 没有返回值类型
- ✓ 必须为所有的变量赋初值
- ✓ 通常要说明为public类型的，即公有的
- ✓ 可以按需包含所需的参数列表

5.2.3 默认的构造方法



- ❑ 每个类都必须至少有一个构造方法。如果程序员没有为类定义构造方法，系统会自动为该类型生成一个默认的构造方法。
- ❑ 默认的构造方法也叫缺省的构造方法。
 - ✓ 缺省构造方法的参数列表及方法体均为空，所生成的对象的属性值也为零或空。
 - ✓ 如果程序员定义了一个或多个构造方法，则将自动屏蔽掉缺省的构造方法。构造方法不能继承。

5.2.3 默认/缺省的构造方法



```
class BankAccount{  
    String ownerName;  
    int accountNumber;  
    float balance;  
}
```

输出结果为:
ownerName=null
accountNumber=0
balance=0.0

```
public class BankTester{  
    public static void main(String args[]){  
        BankAccount myAccount = new BankAccount();  
        System.out.println("ownerName=" + myAccount.ownerName);  
        System.out.println("accountNumber=" +  
myAccount.accountNumber);  
        System.out.println("balance=" + myAccount.balance);  
    }  
}
```

我们可以在调用缺省的构造函数之后直接对其状态进行初始化,
BankAccount myAccount;
myAccount = new BankAccount();
myAccount.ownerName = "Wangli" ;
myAccount.accountNumber = 1000234;
myAccount.balance = 2000.00f;

5.2.4 构造方法重载



构造方法重载(Constructor Overloaded)

□ 在进行

参数调

✓ 在一

✓ 在其

来打

```
public class Student{  
    String name;  
    int age;  
  
    public Student(String s, int n){  
        name = s;  
        age = n;  
    }  
    public Student(String s){  
        this(s, 20);  
    }  
    public Student(){  
        this("Unknown");  
    }  
}
```

的不同的

实现.

字this

5.2.5 finalize()方法



finalize()方法的作用

- ❑ finalize()方法属于Object类，它可被所有类使用。如果对象实例不被任何变量引用时，Java会自动进行“垃圾回收”，收回该实例所占用的内存空间。
- ❑ 在对对象实例进行垃圾收集之前，Java自动调用对象的finalize方法，它相当于C++ 中的析构方法，用来释放对象所占用的系统资源。
- ❑ finalize()方法的说明方式如下：

```
protected void finalize () throws Throwable
```


5.2.6 this引用



- ❑ 类的成员方法中访问类的成员变量，可以使用关键字this指明要操作的对象。
- ❑ 在类方法中，Java自动用this关键字把所有变量和类引用结合在一起。this的使用是不必要的，程序中可以不写该关键字。
- ❑ 有些情况下关键字this是必需的：

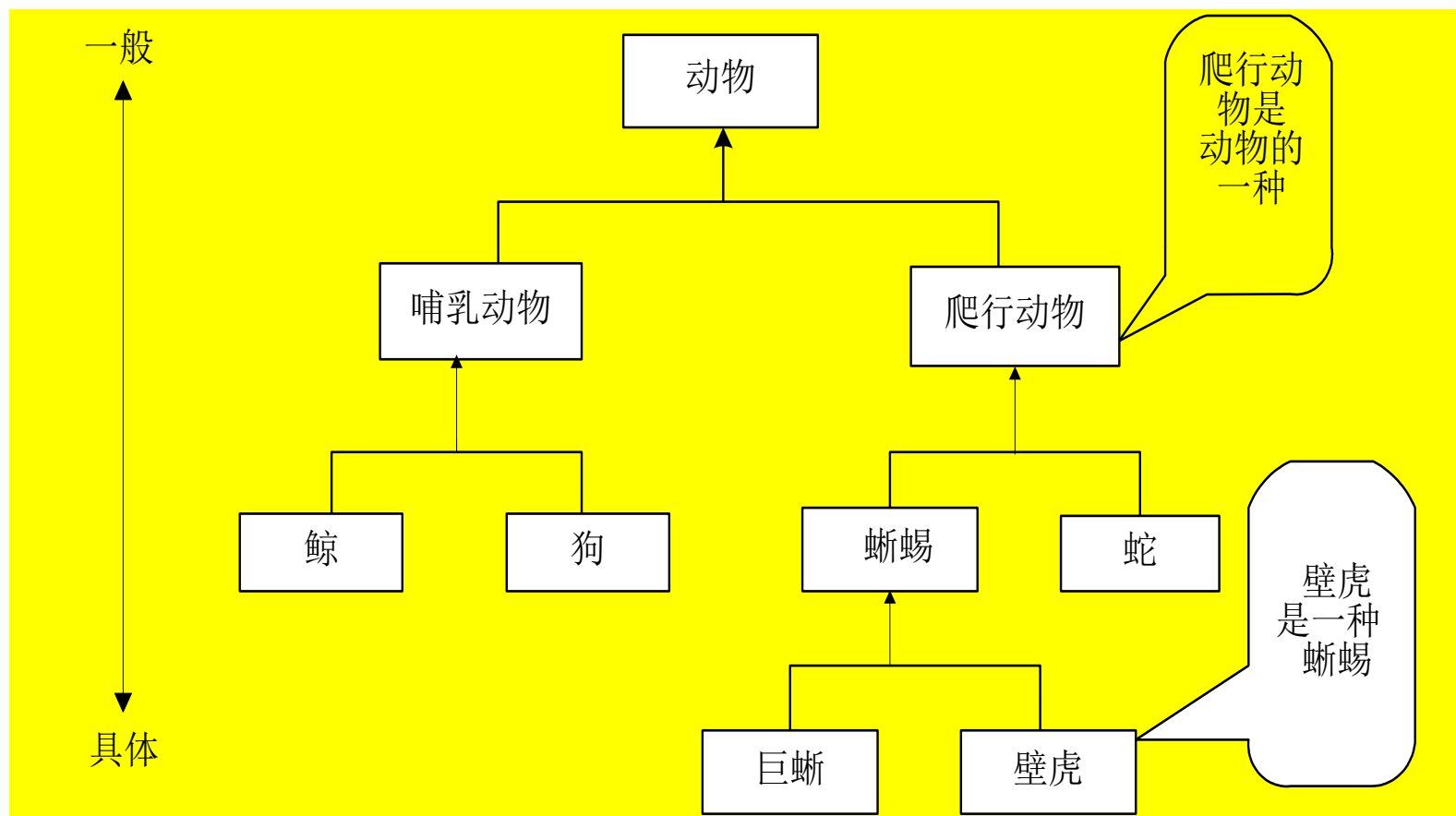
```
✓ public class Date {  
    private int day, month, year;  
    public void printDate() {  
        System.out.println("The current date is (dd / mm / yy): "  
        + this.day + " / " + this.month + " / " + this.year);  
    }  
}
```

一个自变量

5.3 子类



- Java中的类层次结构为树状结构，这与我们在自然界中描述一个事物是类似的。例如我们可以将动物划分为哺乳类动物及爬行类动物，然后又对这两类动物继续细分。



5.3.1 is-a 关系



一般与特殊的关系

- 说雇员Employee，从这个最初的模型派生出多个具体化的版本，如经理Manager。显然，一名Manager首先是一位Employee，它具有Employee的一般特性。除此之外，Manager还有Employee所不具有的额外特性

```
public class Employee {  
    private String name;  
    private Date hireDate;  
    private Date dateOfBirth;  
    private String jobTitle;  
    private int grade;  
    ...  
}
```

```
public class Manager {  
    private String name;  
    private Date hireDate;  
    private Date dateOfBirth;  
    private String jobTitle;  
    private int grade;  
    private String department;  
    private Employee [] subordinates;  
    ...  
}
```

5.3.2 extends关键字



安徽大學
Anhui University

派生

- ❑ 面向对象的语言提供了派生机制，它允许程序员用以前定义的类来定义一个新类。这个新类称作子类，原来的类称作父类或者超类。
- ❑ 两个类中，公共部分的内容方法父类中，特殊的内容放到子类中。父类和子类之间用关键字extends表示派生，其格式如下：

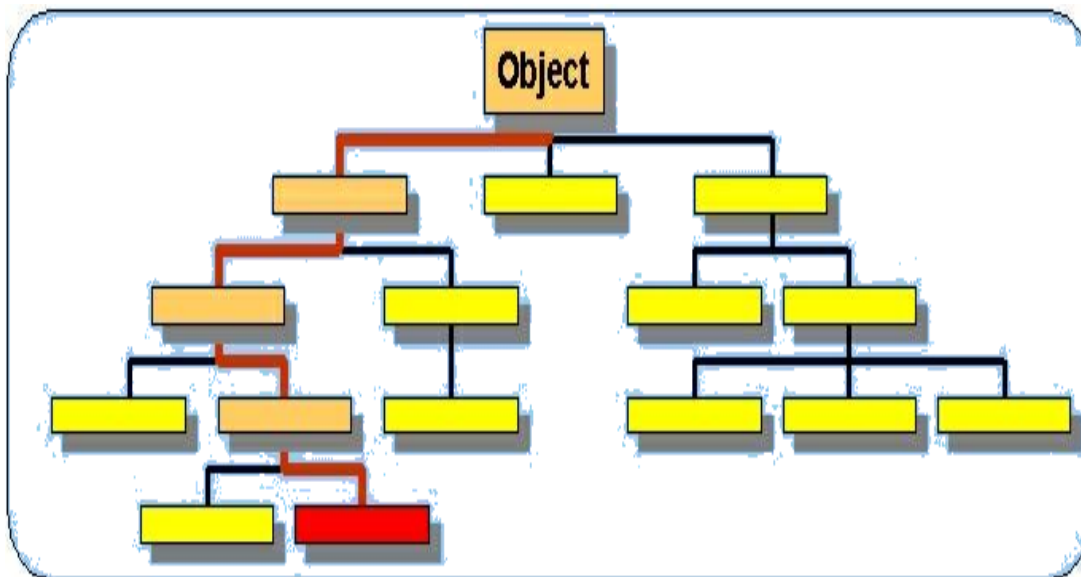
```
public class A extends B { }
```

类A派生于类B，称A为子类，B为父类；如果程序员定义一个类，且没有extends关键字，则默认这个类派生于Object类。**Java中定义的任何类都直接或者间接地派生于Object类。**

5.3.3 单重继承



- ❑ 如果一个类有父类，则其父类只能有一个，Java只允许从一个类中扩展类。这条限制叫单重继承。优点：让代码的可靠性更高。
- ❑ 为了保留多重继承的功能，提出了接口的概念。
- ❑ 一个对象从其所有的父类（在树中通往Object的路径上的类）中继承属性及行为。不能继承构造方法！



5.3.3 单重继承



- ❑ Object类是Java程序中所有类的直接或间接父类，也是类库中所有类的父类，处在类层次最高点。所有其他的类都是从Object类派生出来的，Object类包含了所有Java类的公共属性，其构造方法是Object()
- ✓ **public final Class getClass()**//获取当前对象所属的类信息，返回Class对象。
 - ✓ **public String toString()**//按字符串对象返回当前对象本身的有关信息。
 - ✓ **public boolean equals(Object obj)** //比较两个对象是否是同一对象，是则返回true。
 - ✓ **protected Object clone()** //生成当前对象的一个拷贝，并返回这个复制对象
 - ✓ **Public int hashCode()** //返回该对象的哈希代码值。
 - ✓ **protected void finalize() throws Throwable**//定义回收当前对象时所需完成的资源释放工作。

5.3.3 单重继承



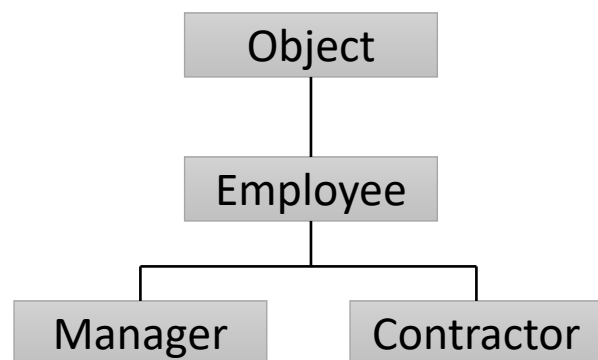
安徽大學
Anhui University

- [例5-11](#) Employee及Manager 类的对象可以使用其父类中定义的公有（及保护）属性和方法，就如同在其自己的类中定义一样
- [例5-12](#) 子类不能直接存取其父类中的私有属性及方法，但可以使用公有（及保护）方法进行存取

5.3.4 转换对象



□ Java允许使用对象之父类类型的一个变量指示该对象，这种现象称为转换对象。



Employee **e** = new Manager() ✓

Manager **m** = new Employee() ✗

子类的实例赋给父类是正确的！

```
public class Employee extends Object
public class Manager extends Employee
public class Contractor extends Contractor
```

例5-13 instanceof 的使用示例

```
public void method (Employee e){
    if ( e instanceof Manager){
    }
    else if ( e instanceof Contractor) {
    }
    else{
    }
}
```


5.3.4 转换对象



安徽大學
Anhui University

□instanceof 运算符：由于类的多态性，类的变量既可以指向本类实例，又可以指向其子类的实例。可以通过instanceof运算符判明一个引用到底指向哪个实例。

5.3.5 方法自变量和异类集合



安徽大學
Anhui University

1. 方法的参量

□ 为了处理的一般性，实例和变量并不总是属于同一类。

```
public TaxRate findTaxRate(Employee e) {  
    // 进行计算并返回e的税率  
}
```

```
// 而在应用程序类中可以写下面的语句  
Manager m = new Manager();  
.....  
TaxRate t = findTaxRate(m);
```

findTaxRate() 的自变量必须是 Employee 类型的，而调用时的参数是 Manager 类型的，但因为 Manager 是一个 Employee，所以可以使用更“一般”的变量类型来调用这个方法。

5.3.5 方法自变量和异类集合



安徽大學
Anhui University

2. 异类集合

- ❑ 异类集合是由不同质内容组成的集合，也就是集合内所含元素的类型不完全一致。
- ❑ 在面向对象的语言中，可以创建有公共祖先类的任何元素的集合。
- ❑ 可以使用Object的一个数组作为任何对象的容器。

```
Employee [] staff = new Employee[1024];  
staff[0] = new Manager();  
staff[1] = new Employee();
```

数组staff中各元素的类型是不相同的。可以像处理同质数组一样，处理由不同类型元素组成的数组。比如，可以对数组元素按年龄大小进行排序等

5.4 方法重写



安徽大學
Anhui University

什么是方法重写?

- 使用类的继承关系，可以从已有的类产生一个新类，在原有特性基础上，增加了新的特性，父类中原有的方法不能满足新的要求，因此需要修改父类中已有的方法。又或者如果子类不需要使用从父类继承来的方法的功能，则可以定义自己的方法。这就是重写（Override）的概念，也称为方法的隐藏。
- 子类中定义方法所用的名字、返回类型及参数表和父类中方法使用的完全一样，称子类方法重写了父类中的方法，从逻辑上看就是子类中的成员方法将隐藏父类中的同名方法。
- 方法重写->语言的多态性

5.4 方法重写



安徽大學
Anhui University

□ 子类重写父类方法多发生在这三种情况下

- ✓ 子类要做与父类不同的事情；在子类中取消这个方法；子类要做比父类更多的事情。

□ 重写的同名方法中，子类方法不能比父类方法的访问权限更严格

- ✓ 如果父类中方法method()的访问权限是public，子类中就不能含有private的method()，否则，会出现编译错误。

5.4 方法重写



安徽大學
Anhui University

```
public class Employee {           程序5-16
    String name;
    int salary;
    public String getDetails() {
        return "Name: " + name + "\n" + "Salary: " + salary;
    }
}

public class Manager extends Employee{
    String department;
    public String getDetails() {
        return "Name: " + name + "\n" + "Manager of " + department;
    }
}
```

5.4 方法重写



安徽大學
Anhui University

程序5-4

```
C:\ 命令提示符

D:\java\program\chapter5>javac Point.java

D:\java\program\chapter5>java Point
This is the superclass!
This is the subclass!
```

程序5-5

在前面定义的二、三维点类基础上，增加求该点到原点距离的方法

```
C:\ 命令提示符

D:\java\program\chapter5\exam5-18>javac Point.java

D:\java\program\chapter5\exam5-18>java Point
p.distance() = 1.4142135623730951
p.distance() = 1.7320508075688772

D:\java\program\chapter5\exam5-18>
```

5.4 方法重写



- ❑ 如果子类已经重写了父类中的方法，但在子类中还想使用父类中被隐藏的方法，可以使用`super`关键字。
- ❑ 在程序5-5的子类Point3d的print()方法中增加一条语句，见程序5-6

A screenshot of a Windows command prompt window titled "命令提示符". The window shows the following commands and output:

```
D:\java\program\chapter5\exam5-19>javac Point.java

D:\java\program\chapter5\exam5-19>java Point
This is the superclass!
This is the subclass!
This is the superclass!

D:\java\program\chapter5\exam5-19>
```


5.4 方法重写



安徽大學
Anhui University

重写和重载的区别

- 重写发生在父类和子类之间，子类继承父类，子类中与父类相同的方法名。
- 重载发生在相同的类中，在相同类中有多个相同的方法名，不同的参数列表。

应用重写的规则

- 重写方法的允许访问范围不能小于原方。
- 重写方法所抛出的异常不能比原方法更多。

5.4 方法重写



安徽大學
Anhui University

程序5-17 方法重写的访问权限示例

```
class SuperClass{  
    public void method(){  
        .....  
    }  
}  
class SubClass extends SuperClass{  
    private void method(){  
        .....  
    }  
}  
public class Test{  
    public static void main(String args[]){  
        SuperClass s1 = new SuperClass();  
        SuperClass s2 = new SubClass();  
        s1.method();  
        s2.method();  
    }  
}
```

编译后会出现以下错误信息：

Test.Java:6: Methods can't be overridden to be more private. Method void method() is public in class SuperClass.
private void method(){

5.4 方法重写



安徽大學
Anhui University

程序5-18 方法重写的异常处理示例

```
import java.io.*;
```

```
class Parent{
```

```
    void method(){
```

```
    }
```

```
}
```

```
class Child extends Parent{
```

```
    void method() throws IOException{
```

```
    }
```

```
}
```

编译后会出现以下错误信息：

Child.Java:8: Invalid exception class
Java.io.IOException in throws clause. The
exception must be a subclass of an exception
thrown by void method() from class Parent.
void method() throws IOException{

5.4 方法重写



安徽大學
Anhui University

父类构造方法调用

- ❑ 一个父类的对象要在子类运行前完全初始化。
- ❑ `super`关键字也可以用于构造方法中，其功能为调用父类的构造方法。子类不能从父类继承构造方法在子类的构造方法中调用某一个父类构造方法
- ❑ 如果在子类的构造方法的定义中没有明确调用父类的构造方法，则系统自动调用父类的缺省构造方法（即无参数的构造方法）
- ❑ 调用了父类的构造方法语句必须出现在子类构造方法的第一行

5.4 方法重写



```
class Employee{
    String name;
    public Employee(String s){
        name = s;
    }
}
class Manager extends Employee{
    String department;
    public Manager(String s, String d){
        super(s);
        department = d;
    }
}
```

```
department = d;
super(s);
```



5.5 多态



安徽大學
Anhui University

- ❑ “多态”，源自希腊语，意思是“多种形式”。多态是面向对象编程的一个非常重要的概念。
- ❑ 举例：假定Employee类中有一个getDetails()方法，而由Employee派生的Manager类中亦有一个同名、同参数、同返回类型的getDetails()方法。由于重写的两个getDetails()方法可能执行了不同的代码内容，这就是多态的问题。
- ❑ 方法的重载和重写，都可以看成多态的表现形式。

5.5 多态



- ❑ 在Java中，多态允许同一条程序指令在不同的上下文中意味着不同的事情。具体来说，一个方法名当作一条指令时，依据执行这个动作的对象类型，可能导致不同的动作。
- ❑ 变量的静态类型（static type）是出现在声明中的类型。静态类型是在代码编译时固定且确定下来的。运行时某一时刻变量指向的对象的类型称为动态类型（dynamic type）。变量的动态类型随运行进程会改变。引用类型的变量称为多态变量（polymorphic variable），因为执行过程中，它的动态类型可以不同于静态类型，且可改变。
- ❑ 调用稍后可能被重写的方法的这种处理方式，称为动态绑定（dynamic binding）或后绑定（late binding）。

5.6 JAVA包



安徽大學
Anhui University

- 一个Java源代码文件称为一个编译单元。
- 一个编译单元中只能有一个public类，且该类名与文件名相同。每个类都产生一个 .class文件。
这种机制下，不同名的类中如果含有相同名字的方法或成员变量，不会造成混淆.但由于有继承机制，有可能程序员用到了其他人定义的类，比如是从互联网上下载类，使用过程中又不知晓全部的类名，这就有冲突的可能了。
- 所以在Java中必须要有一种对名字空间的完全控制机制，以便可以建立一个唯一的类名。这就是包机制，用于类名空间的管理。

5.6.1 JAVA包的概念



- ❑ 包是类的容器，包的设计人员利用包来划分名字空间，用于分隔类名空间，以避免类名冲突。没有包定义的源代码文件成为未命名的包中的一部分，在未命名的包中的类不需要写包标识符。
- ❑ 使用package指明源文件中的类属于哪个具体的包。包语句的格式为：
`package pkg1[.pkg2[.pkg3...]];`
- ❑ package语句一定是源文件中的第一条可执行语句，它的前面只能有注释或空行。一个文件中最多只能有一条package语句
- ❑ 包的名字有层次关系，各层之间以点分隔。包层次必须与Java开发系统的文件系统结构相同。通常包名中全部用小写字母
- ❑ 一个包可以包含若干个类文件，还可包含若干个包。一个包要放在指定目录下，通常用classpath指定搜寻包的路径。包名本身对应一个目录（用一个目录表示）

5.6.1 JAVA包的概念



安徽大學
Anhui University

- `package java.awt.image;`
- 在Windows系统下，此文件必须存放在`java\awt\image`目录下；如果在unix系统下，文件须放在`java/awt/image`目录下。此定义语句说明当前的编译单元是包`java.awt.image`的一部分，文件中的每一个类名前都有前缀`java.awt.image`，因此不再会有重名问题

5.6.2 import语句



□ 使用MyClass类，或mypackage包中的其它public类，则需要使用使用全名。

```
mypackage.MyClass m = new mypackage.MyClass();
```

□ 也可以先使用import语句引入所需要的类，程序中无需再使用全名。

```
import mypackage.*;  
//...  
MyClass m = new MyClass();
```



```
package mypackage;  
public class MyClass {  
    //...  
}
```

5.6.2 import语句



□ **import**语句只用来将其他包中的类引入当前名字空间中，程序中不需再引用同一个包或该包的任何元素，而当前包总是处于当前名字空间中

□ 引入语句的格式如下

- ◆ `import pkg1[.pkg2[.pkg3...]].(类名|*)`
- ◆ “*” 表示引入所有类

5.6.2 import语句



安徽大學
Anhui University

例5-20 假设有一个包a，在a中的一个文件内定义了两个类xx和yy，其格式如下：

```
package a;  
class xx{  
...  
}  
class yy{  
...  
}
```

当在另外一个包b中的文件zz.java中使用a中的类时，语句形式如下：

```
// zz.java  
package b;                //说明是在包b中  
  
import a.*;               //引入a中的全部类  
class zz extends xx {  
    yy y;                  //使用的是a中的yy类  
    ...  
}
```

5.6.3 CLASSPATH环境变量



安徽大學
Anhui University

- 环境变量classpath将指示着javac编译器如何查找所需要的对象
- 在编译命令中使用-d选项，则Java编译器可以创建包目录，并把生成的类文件放到该目录中

```
c:\>javac -d destpath Test.java
```

则编译器会自动在destpath目录下建立一个子目录p1，
并将生成的.class文件都放到destpath/p1下

5.6.4 访问权限与数据隐藏



安徽大學
Anhui University

- day、month和year是private的，这意味着只能在Date类中的方法内访问这些成员，而在类外的方法中不能访问它们，这就是访问权限(Access Control).

```
Date d = new Date();  
d.day = 32;  
d.month = 2; d.day = 30; // 语法正确但语义错误  
d.month = d.month + 1; // 没有进行月份的循环检查
```

```
public void setDay( int targetDay ){  
    if ( targetDay > this.DaysInMonth() ) {  
        System.err.println ( “ 输入的天数不合法” );  
    }  
    else{  
        this.day = targetDay;  
    }  
}
```

- 类的成员不建议设为公有，直接供外界随意更改。如果是私有的成员，则可以借助方法来访问成员变量。这就是接下来要介绍的封装的问题。

5.6.5 封装



安徽大學
Anhui University

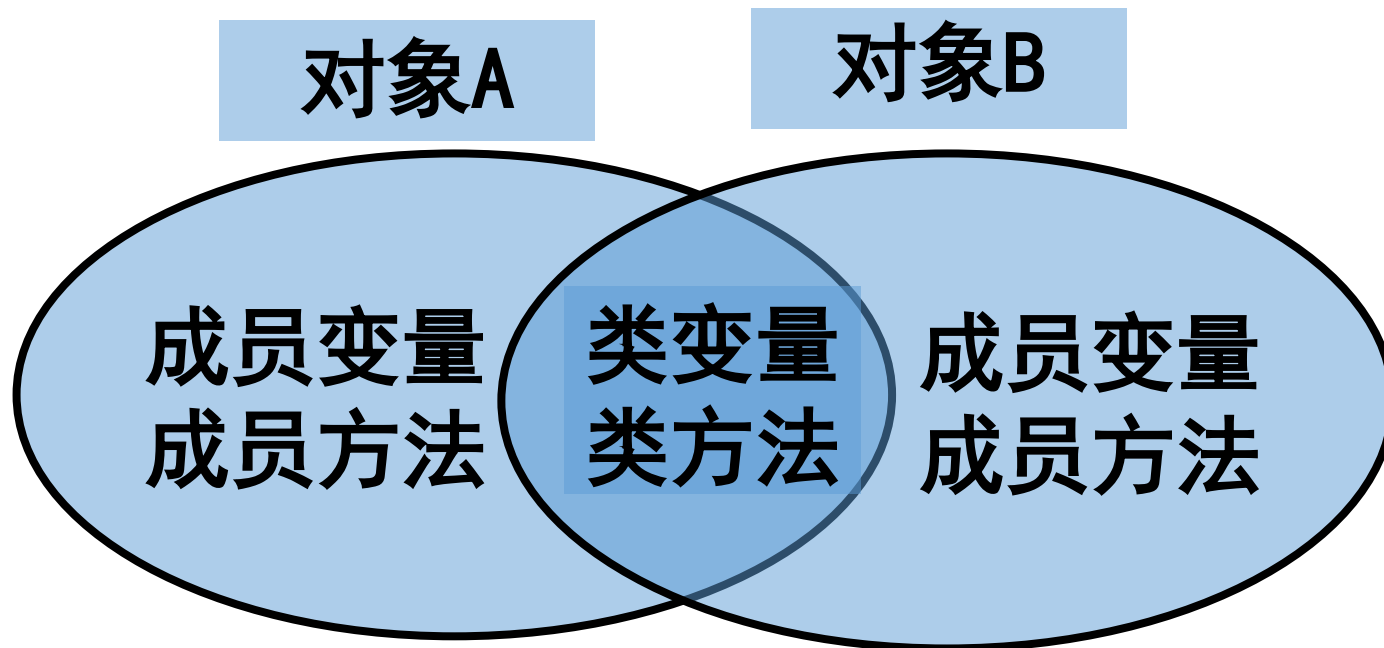
- ❑除了保护对象的数据不受非法修改的外，强制使用者通过方法来访问数据是确保方法调用后各数据仍合法的更简单的方法。
- ❑如果数据的访问时完全开放的，类的每个使用者都可以修改day值，并需要时刻记住day值的变化而导致day-month-year之间的不一致，从而导致程序混乱。
- ❑因此，通过方法来访问数据，类成员，更具安全性，这就是封装。
- ❑封装是面向对象一个重要原则。
- ❑封装的两层含义：（1）将对象的全部属性数据和对象的全部操作结合在一起，形成一个统一体；（2）是尽可能地隐藏对象内部细节，只保留对外的有限接口；

5.7 类成员



安徽大學
Anhui University

- 类成员，它包括类变量和类方法。它是不依赖于特定对象的内容。
- 不同对象的成员其内存地址是不同的。
- 系统只在实例化类的第一个对象的时候，为类成员分配内存，以后再生成该类的实例对象时，将不再为类成员分配内存，不同对象的类变量将共享同一内存空间。



5.7.1 类变量



- 让一个变量被类的多个实例对象所共享，以实现多个对象之间的通信，或用于记录已被创建的对象个数，这样的变量有时也被称为类变量（或静态变量）
- 将一个变量定义为类变量的方法就是将这个变量标记上关键字 **static**
- 程序5-8 类变量（或静态变量）的作用

```
命令提示符
D:\java\program\chapter5>
D:\java\program\chapter5>
Count.counter is 0
Tom's serialNumber is 1
John's serialNumber is 2
Now Count.counter is 2
D:\java\program\chapter5>
```

在这个例子中，每一个被创建的对象得到一个唯一的serial number，这个号码由初始值1开始递增。由于变量counter被定义为类变量，为所有对象所共享，因而当一个对象的构造方法将其递增1后，下一个将要被创建的对象所看到的counter值就是递增之后的值

5.7.1 类变量



□Java语言中没有全局变量的概
量。

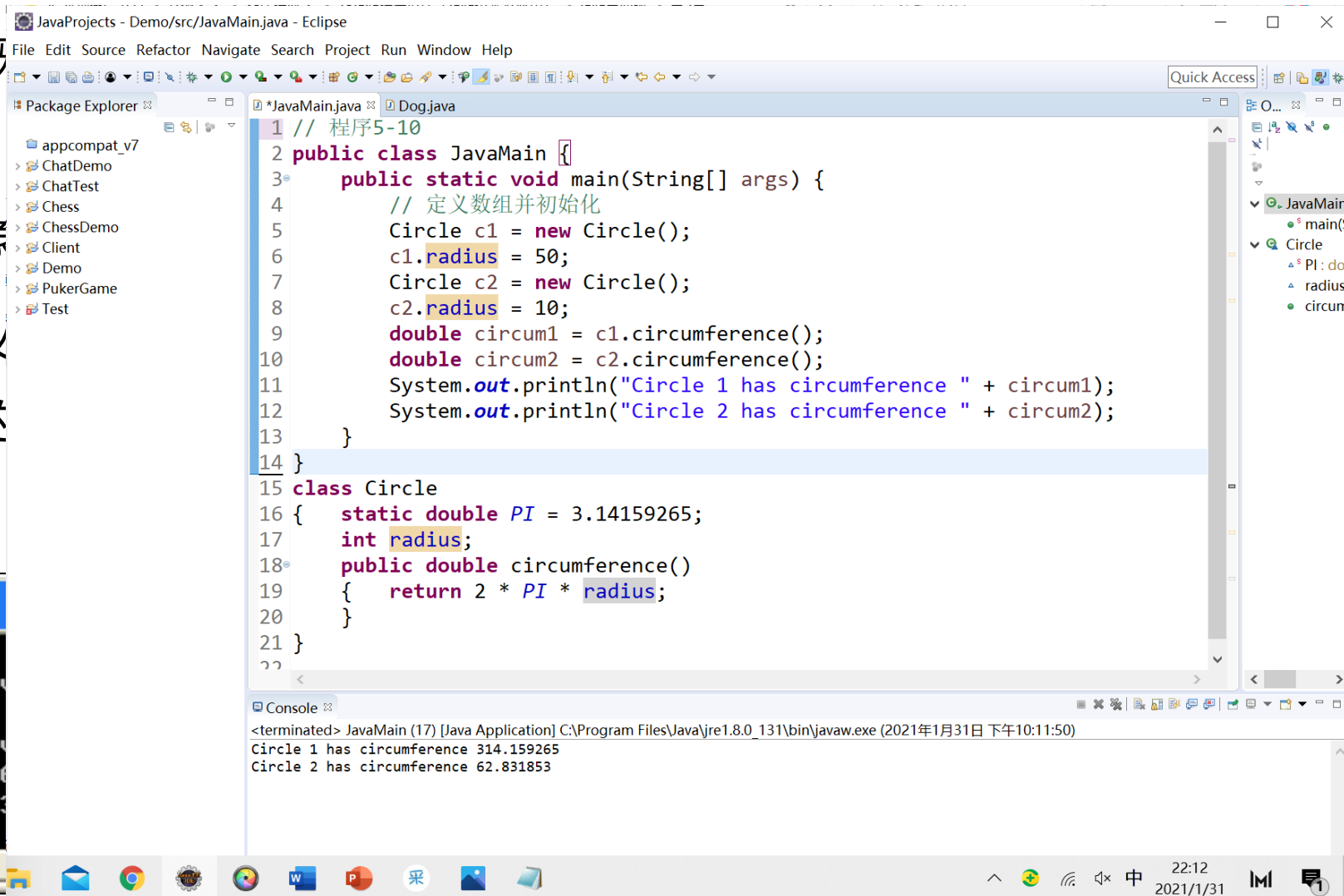
□类变量是唯一为类中所有对象

□如果一个类变量同时还被定义
量时甚至无须生成一个该类的

□程序5-9

```
C:\ 命令提示符

D:\java\program\chapter5\prog5-7>jav
D:\java\program\chapter5\prog5-7>jav
Circle 1 has circumference 314.15926
Circle 2 has circumference 62.831853
```



5.7.2 类方法



安徽大學
Anhui University

□ 标记上关键字static的方法称为类方法（或称静态方法）

□ 程序5-11

```
JavaProjects - Demo/src/JavaMain.java - Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
  appcompat_v7
  > ChatDemo
  > ChatTest
  > Chess
  > ChessDemo
  > Client
  > Demo
  > PukerGame
  > Test

JavaMain.java Dog.java
1 // 程序5-11
2 public class JavaMain {
3     public static void main(String[] args) {
4         // 定义数组并初始化
5         int a = 9;
6         int b = 10;
7         int c = GeneralFunction.addUp(a,b);
8         System.out.println("addUp() gives" + c);
9     }
10 }
11
12 class GeneralFunction
13 {
14     public static int addUp(int x, int y)
15     {
16         return x+y;
17     }
18 }

Console
<terminated> JavaMain (17) [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (2021年1月31日 下午10:08:21)
addUp() gives19
```

5.7.2 类方法



安徽大學
Anhui University

□程序5-25

```
public class Converter {  
    public static int centigradeToFahrenheit(int cent){  
        return (cent * 9 / 5 + 32);  
    }  
}
```

□调用类方法时，前缀使用是的类名，而不是对象实例名，如下所示：

✓ Converter.centigradeToFahrenheit(28);

□如果从当前类中的其它方法中调用，则不需要写类名，可以直接写方法名：

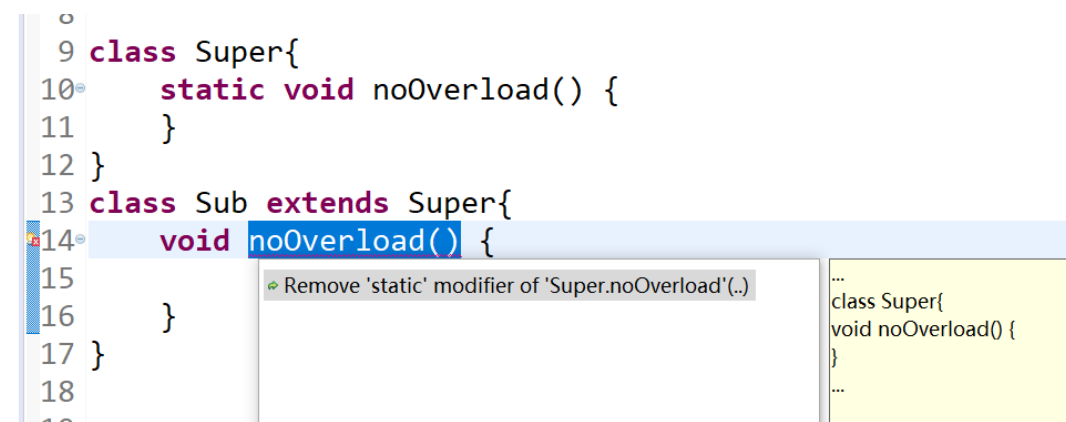
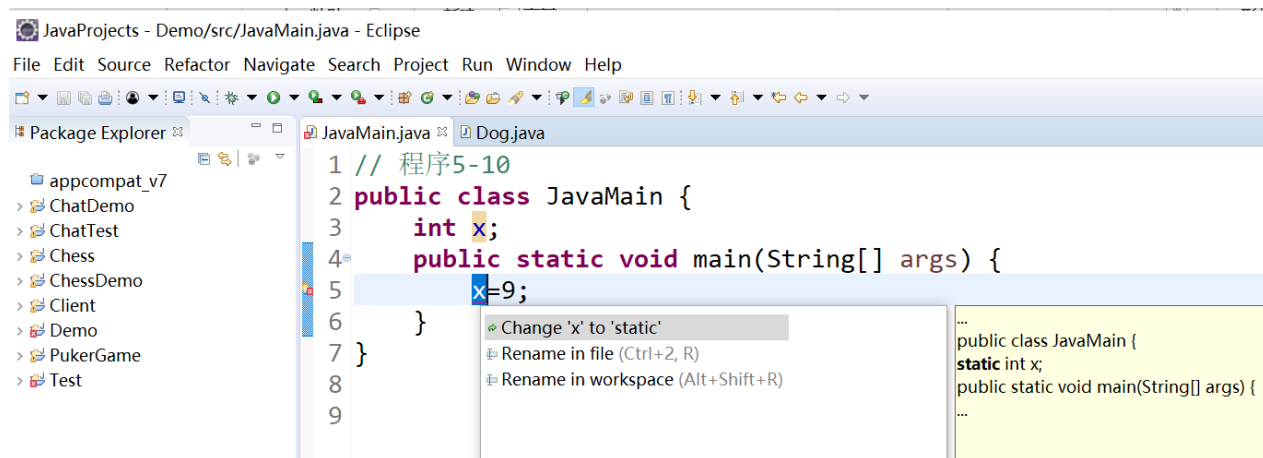
✓ centigradeToFahrenheit(28)

5.7.2 类方法



静态方法

- 调用类方法时，前缀使用是的类名，而不是对象实例名，如下所示：
- 由于静态方法可以在没有定义它所从属的类的对象的情况下加以调用，故不存在`this`值
- 静态方法不能被重写。也就是说，在这个类的子孙类中，不能有相同名称、相同参数的方法。



5.8 关键字final



- 它既可以用来修饰一个类，也可用于修饰类中的成员变量或成员方法。
- 用这个关键字进行修饰的类或类的成员都是不能改变的。
- 如果一个方法被定义为final，则不能被重写；
- 如果一个类被定义为final，它不能有子类。
- 被标记为final的类将不能被继承，这样的类可以称之为**终极类**（final class）其声明的格式为：

```
final class finalClassName{  
    .....  
}
```

5.8 关键字final



```
final public class FinalClass{  
    int memberar;  
    void memberMethod(){};  
}
```

在编译时会出现以下错误信息：
SubFinalClass.Java:7: Can't subclass
final classes: class FinalClass
class SubFinalClass extends FinalClass{

```
class SubFinalClass extends FinalClass{  
    int submembervar;  
    void subMemberMethod(){};  
}
```


5.8 关键字final



安徽大學
Anhui University

终极方法

- 它既可以用来修饰一个类，也可用于修饰类中的成员变量或成员方法。
- 成员方法被标记为final成为终极方法（final method），被标记final的方法将不能被重写
 - ✓ 安全考虑。这样调用final类型的方法时可以确保被调用的是正确的、原始的方法
 - ✓ 标记为final有时也被用于优化
- 终极方法的定义格式为：

```
final returnType finalMethod([paramlist]){  
    .....  
}
```

5.8 关键字final



安徽大學
Anhui University

```
class FinalMethodClass{
    final void finalMethod (){
        ...                //原程序代码
    }
}

class OverloadClass extends FinalMethodClass{
    void finalMethod(){ // 错误!
        ...                //子程序代码
    }
}
```

5.8 关键字final



终极变量

□ 一个变量被标记为final，则会使它成为一个常量。

```
class Const{  
    final float PI = 3.14f;  
    final String language = "Java";  
}
```

```
public class UseConst{  
    public static void main(String args[]){  
        Const myconst = new Const();  
        myconst.PI=3.1415926f;  
    }  
}
```

在编译时会出现以下错误信息：
UseConst.Java:9: Can't assign
a value to a final variable:
PI
myconst.PI=3.1415926f;

5.8 关键字final



终极引用

- 将程序中可能用到的一系列常量定义在一个类中，其他类通过引入该类来直接使用这些常量，保证常量使用的统一，修改提供方便
- 将一个引用类型的变量标记为final，那么这个变量将不能再指向其他对象，但它所指对象的取值仍然是可以改变的

```
class Car{
    int number=1234;
}
class FinalVariable{
    public static void main(String args[]){
        final Car mycar = new Car();
        mycar.number = 8888; //可以!
        mycar = new Car(); //错误!
    }
}
```

5.9 抽象类



安徽大學
Anhui University

- ❑ 定义了方法但没有定义具体实现的类通常称为抽象类 (abstract class)
- ❑ 通过关键字abstract把一个类定义为抽象类
- ❑ 每一个未被定义具体实现的方法也应标记为abstract (可以称为抽象方法)
- ❑ 只有抽象类才能具有抽象方法：抽象类中可以由抽象方法和非抽象方法；非抽象类中不能声明抽象方法。
- ❑ 不能用抽象类作为模板来创建对象，必须生成抽象类的一个非抽象的子类后才能创建实例
- ❑ 如果一个抽象类除了抽象方法外什么都没有，则使用接口更合适

5.9 抽象类



安徽大學
Anhui University

□抽象类的定义如下：

```
public abstract class Shape {  
    // 定义体  
}    //为使此类有用，它必须有子类
```

□抽象方法的定义：

```
public abstract <returnType> <methodName> (参数列表);
```

5.9 抽象类



安徽大學
Anhui University

```
abstract class ObjectStorage{
    int objectnum=0;
    Object storage[] = new Object[100];
    abstract void put(Object o); //注意: 没有大括弧("{}")
    abstract Object get();
}

class Stack extends ObjectStorage
{
    private int point=0;
    public void put(Object o) {
        storage[point++] = o;
        objectnum++;
    }
    public Object get()
    {
        objectnum--;
        return storage[--point];
    }
}
```

```
class Queue extends ObjectStorage
{
    private int top=0;
    private int bottom=0;

    public void put(Object o)
    {
        storage[top++] = o;
        objectnum++;
    }
    public Object get()
    {
        objectnum--;
        return storage[bottom++];
    }
}
```

5.10 接口



安徽大學
Anhui University

- ❑ 接口 (interface) 是抽象类功能的另一种实现方法, 可将其想象为一个“纯”的抽象类。
- ❑ 允许创建一个类的基本形式, 包括方法名、自变量列表以及返回类型, 但不规定方法主体。
- ❑ 接口中所有的方法都是抽象方法, 都没有方法体。
- ❑ Java通过允许一个类实现 (implements) 多个接口从而实现了比多重继承更加强大的能力, 并具有更加清晰的结构。

5.10 接口



安徽大學
Anhui University

□接口的定义形式为:

```
[接口修饰符] interface 接口名称 [extends 父类名]{  
    .....    //方法原型或静态常量  
}
```

例如: `public interface Insurable extends SuperInsurable { ..... }`

接口本身也具有数据成员与方法，但数据成员一定要赋初值，且此值将不能再更改

5.10 接口



- ❑ 在接口中定义的成员变量都缺省为终极类变量，即系统会将其自动增加final和static这两个关键字，并且对该变量必须设置初值

```
interface CharStorage{  
    void put(char c);  
    char get();  
}
```

```
public interface Insurable{  
    public int getNumber();  
    public int getCoverageAmount();  
    public double calculatePremium();  
    public Date getExpiryDate();  
}
```

5.10 接口



接口的实现

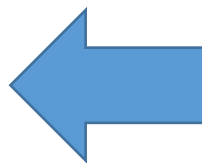
- ❑ 实现接口的类不从该接口的定义中继承任何行为，在实现该接口的类的任何对象中都能够调用这个接口中定义的方法。在实现的过程中，这个类还可以同时实现其他接口
- ❑ 要实现接口，可在一个类的声明中用关键字implements表示该类已经实现的接口。完成接口的类必须实现接口中的所有抽象方法
- ❑ Implements语句的格式如下：

```
public class className implements interfaceName {  
    /* Bodies for the interface methods */  
    /* Own data and methods. */  
}
```

5.10 接口



```
public class Car implements Insurable {  
    public int getPolicyNumber() {  
        // write code here  
    }  
    public double calculatePremium() {  
        // write code here  
    }  
    public Date getExpiryDate() {  
        // write code here  
    }  
    public int getCoverageAmount() {  
        // write code here  
    }  
}
```



```
public interface Insurable{  
    public int getNumber();  
    public int getCoverageAmount();  
    public double calculatePremium();  
    public Date getExpiryDate();  
}
```

5.10 接口



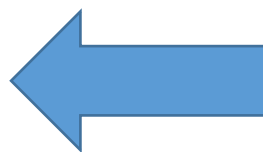
安徽大學
Anhui University

```
class Stack implements CharStorage{  
    private char mem[] = new char[10];  
    private int point = 0;
```

```
    void put(char c){  
        mem[point] = c;  
        point++;  
    }
```

```
    char get(){  
        point--;  
        return mem[point];  
    }
```

```
}
```



```
interface CharStorage{  
    void put(char c);  
    char get();  
}
```

5.10 接口



安徽大學
Anhui University

接口的实现

- ❑ 不能直接由接口来创建对象，必须通过由实现接口的类来创建。
- ❑ 同抽象类一样，使用接口名称作为一个引用的类型也是允许的，即可以声明接口类型的变量（或数组），并用它访问对象。

5.11 内部类



安徽大學
Anhui University

接口的实现

- ❑ 不能直接由接口来创建对象，必须通过由实现接口的类来创建。
- ❑ 类名只能在定义的范围内被使用，内部类的名称必须区别于外部类。
- ❑ 内部类可以使用外部类的类变量和实例变量，也可使用外部的局部变量。
- ❑ 内部类可以定义为abstract类型。
- ❑ 内部类也可以是一个接口，这个接口必须由另一个内部类来实现。

5.11 内部类



安徽大學
Anhui University

接口的实现

- ❑ 内部类可以被定义为private或protected类型。当一个类中嵌套另一个类时，访问保护并不妨碍内部类使用外部类的成员
- ❑ 被定义为static型的内部类将自动转化为顶层类，它们不能再使用局部范围中或其他内部类中的数据和变量
- ❑ 内部类不能定义static型成员，而只有顶层类才能定义static型成员。如果内部类需要使用static型成员，这个成员必须在外部类中加以定义
- ❑ 应用内部类的好处在于它是创建事件接收者的一个方便的方法

5.11 内部类



安徽大學
Anhui University

//程序5-14

```
import java.awt.*;
import java.awt.event.*;

public class TwoListenInner
{
    private Frame f;
    private TextField tf;

    public static void main(String ars[])
    {
        TwoListenInner that = new TwoListenInner();
        that.go();
    }

    public void go()
    {
        f = new Frame("Two listeners example");
        f.add("North", new Label("Click and drag the mouse"));
        tf=new TextField (30);
        f.add("South", tf);
        f.addMouseMotionListener(new MouseMotionHandler());
        f.addMouseListener(new MouseEventHandler());
        f.setSize(300, 200);
        f.setVisible(true);
    }
}
```

// MouseMotionHandler 为一个内部类

```
    public class MouseMotionHandler extends MouseMotionAdapter
    {
        public void mouseDragged (MouseEvent e)
        {
            String s="Mouse dragging: X="+e.getX()
            +"Y="+e.getY();
            tf.setText(s);
        }
    }

    // MouseEventHandler
    public class MouseEventHandler extends MouseAdapter
    {
        public void mouseEntered(MouseEvent e)
        {
            String s = "The mouse entered";
            tf.setText(s);
        }

        public void mouseExited (MouseEvent e)
        {
            String s = "The mouse has left the building";
            tf.setText(s);
        }
    }
}
```

5.11 内部类



匿名类

- 在定义一个类的时候，定义了一个匿名类
- 匿名类使代码更简洁
- 使用匿名类重写了父类的方法

是在定义

//程序5-15

```
public void go()
{
    f = new Frame("Two listeners example");
    f.add("North", new Label("Click and drag the mouse"));
    tf = new TextField(30);
    f.add("South", tf);

    f.addMouseMotionListener(new MouseMotionAdapter(){
        public void mouseDragged(MouseEvent e)
        {
            String s = "Mouse dragging: X=" + e.getX() + "Y=" + e.getY();
            tf.setText(s);
        }
    }); // <- 注意括号在这里的用法

    f.addMouseListener(new MouseEventHandler());
    f.setSize(300, 200);
    f.setVisible(true);
}
```

5.11 内部类



安徽大學
Anhui University

内部类的工作方式

- ❑ 内部类可以访问其外部类，因此JDK1.1编译器给内部类另外添加了一个private成员变量和一个构造方法对它进行初始化
- ❑ 编译器还自动使外部类的范围适用于内部类，将 “.” 替换为 “\$” 。这样编译器看到的改动后的源代码为程序5-16

java中的内部类和接口加在一起，可以解决常被**C++**程序员抱怨**java**中存在的一个问题 没有多继承。实际上，**C++**的多继承设计起来很复杂，而**java**通过内部类加上接口，可以很好的实现多继承的效果。

5.12 包装类



❑ 当你想用处理对象一样的方法来处理基本类型的数据时，必须将基本类型值“包装”为一个对象，为此，Java提供了包装类。

❑ 包装（wrapper）类表示一种特殊的基本类型。

➤ 例如，Integer类表示一个普通的整型量。

➤ `Integer ageObj = new Integer(40);`

基本数据类型	包装类
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean
void	Void

5.12 包装类



❑ Integer (int value)

- 构造方法：创建新的Integer对象，用来保存value的值。

❑ byte byteValue ()、double doubleValue ()、float floatValue ()、int intValue ()、long longValue ()

- 按对应的基本数据类型返回Integer对象的值。

❑ static int parseInt (String str)

- 按int类型返回存储在指定字符串str中的值。

❑ static String toBinaryString (int num)、static String toHexString (int num)、static String toOctalString (int num)

- 将指定的整型值在对应的进制下按字符串形式返回。

❑ Integer类有两个常量MIN_VALUE和MAX_VALUE，它们分别保存int型中的最小值及最大值。

❑ 其他的包装类中也有对应于相应类型的类似的常量。

5.12 包装类



安徽大學
Anhui University

装箱与拆箱

□ 自动将基本数据类型转为对应的包装类的过程称为自动装箱 (Autoboxing)

□ 逆向的转换称为拆箱 (unboxing)

- `Integer obj2 = new Integer(69);`
- `int num2;`
- `num2 = obj2;` `// 自动解析出int型`

5.13 习题



安徽大學
Anhui University

本章内容很重要，建议完成课后所有习题。

本章讲课结束

谢 谢！